

Google Colab Tutorial: Land Use Land Cover Classification

Getting Started with LULC Classification in Google Colab

This tutorial will guide you through performing Land Use Land Cover (LULC) classification using Google Colab, covering both traditional machine learning and deep learning approaches.

Prerequisites

- Google account (for Colab access)
- Basic understanding of Python
- Familiarity with machine learning concepts

Part 1: Setting Up Your Environment

1.1 Open Google Colab

1. Go to [Google Colab](#)
2. Sign in with your Google account
3. Create a new notebook: **File > New Notebook**

1.2 Install Required Libraries

```
python
```

```
# Install geospatial and remote sensing libraries
!pip install rasterio
!pip install geopandas
!pip install earthpy
!pip install scikit-learn
!pip install torch torchvision
!pip install matplotlib seaborn
!pip install opencv-python

# For Google Earth Engine (if using GEE approach)
!pip install earthengine-api

# Import essential libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import torch
import torch.nn as nn
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
import rasterio
import warnings
warnings.filterwarnings('ignore')

print("✅ Libraries installed and imported successfully!")
```

Part 2: Understanding LULC Classification

2.1 Common LULC Classes

We'll work with these fundamental land cover types:

- **Urban/Built-up:** Cities, towns, infrastructure
- **Forest:** Dense vegetation, trees
- **Cropland:** Agricultural areas
- **Water:** Rivers, lakes, oceans
- **Barren:** Bare soil, rocks, sand
- **Grassland:** Pastures, natural grass areas

2.2 Data Sources

```
python

# Common satellite data sources for LULC classification
print("📡 Popular Satellite Data Sources:")
print("1. Landsat 8/9 (30m resolution)")
print("2. Sentinel-2 (10m resolution)")
print("3. MODIS (250m-1km resolution)")
print("4. Pre-processed datasets: EuroSAT, BigEarthNet")
```

Part 3: Method 1 - Using Pre-trained Models with PyTorch

3.1 Download Sample Dataset

```
python

# We'll use the EuroSAT dataset - a collection of 27,000 Sentinel-2 images
# covering 13 spectral bands and 10 LULC classes

import urllib.request
import zipfile
import os

# Create data directory
os.makedirs('data', exist_ok=True)

print("📦 For this tutorial, we'll use sample data...")
print("In practice, you would download datasets like:")
print("- EuroSAT: 10 LULC classes from Sentinel-2")
print("- Sen12MS: Multi-sensor dataset")
print("- BigEarthNet: Large-scale dataset")

# Sample class names
class_names = [
    'AnnualCrop', 'Forest', 'HerbaceousVegetation',
    'Highway', 'Industrial', 'Pasture',
    'PermanentCrop', 'Residential', 'River', 'SeaLake'
]

print(f"📋 Working with {len(class_names)} LULC classes:")
for i, class_name in enumerate(class_names):
    print(f" {i+1}. {class_name}")
```

3.2 Create a Simple CNN Model

python

```
import torch.nn as nn
import torch.nn.functional as F

class LULCClassifier(nn.Module):
    def __init__(self, num_classes=10, num_bands=3):
        super(LULCClassifier, self).__init__()

        # Convolutional layers
        self.conv1 = nn.Conv2d(num_bands, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)

        # Pooling and dropout
        self.pool = nn.MaxPool2d(2, 2)
        self.dropout = nn.Dropout(0.5)

        # Fully connected layers (adjust based on input size)
        self.fc1 = nn.Linear(128 * 28 * 28, 512) # For 224x224 input
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))

        x = x.view(-1, 128 * 28 * 28) # Flatten
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.dropout(F.relu(self.fc2(x)))
        x = self.fc3(x)

        return x

# Initialize model
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = LULCClassifier(num_classes=10, num_bands=3).to(device)

print(f"🔑 Model created and moved to: {device}")
print(f"📊 Model parameters: {sum(p.numel() for p in model.parameters());}")
```

3.3 Training Setup

python

```
import torch.optim as optim

# Training parameters
learning_rate = 0.001
num_epochs = 20
batch_size = 32

# Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Learning rate scheduler
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

print("🤖 Training configuration:")
print(f"  Learning Rate: {learning_rate}")
print(f"  Epochs: {num_epochs}")
print(f"  Batch Size: {batch_size}")
print(f"  Optimizer: Adam")
print(f"  Loss Function: CrossEntropyLoss")
```

Part 4: Method 2 - Traditional Machine Learning Approach

4.1 Feature Extraction Function

```
python
```

```

def extract_features(image_array):
    """
    Extract spectral and textural features from satellite imagery
    """
    features = []

    # Spectral features (mean, std for each band)
    for band in range(image_array.shape[2]):
        band_data = image_array[:, :, band]
        features.extend([
            np.mean(band_data),
            np.std(band_data),
            np.min(band_data),
            np.max(band_data)
        ])

    # Additional spectral indices (example for RGB)
    if image_array.shape[2] >= 3:
        red = image_array[:, :, 0]
        green = image_array[:, :, 1]
        blue = image_array[:, :, 2]

        # Normalized Difference Vegetation Index (NDVI) approximation
        ndvi_like = (green - red) / (green + red + 1e-8)
        features.extend([
            np.mean(ndvi_like),
            np.std(ndvi_like)
        ])

    return np.array(features)

print("🔍 Feature extraction function ready!")
print("Features include: spectral statistics + vegetation indices")

```

4.2 Machine Learning Models Setup

python

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Initialize models
models = {
    'Random Forest': RandomForestClassifier(
        n_estimators=100,
        random_state=42,
        n_jobs=-1
    ),
    'SVM': SVC(
        kernel='rbf',
        random_state=42,
        probability=True
    )
}

print("🤖 Machine Learning Models:")
for name, model in models.items():
    print(f"✓ {name}: {type(model).__name__}")
```

Part 5: Method 3 - Using Google Earth Engine in Colab

5.1 Authenticate Google Earth Engine

```
python
```

try:

```
import ee
```

```
# Authenticate and initialize Earth Engine
```

```
# First time: will prompt for authentication
```

```
ee.Authenticate()
```

```
ee.Initialize()
```

```
print("🌐 Google Earth Engine initialized successfully!")
```

```
# Test with a simple image collection
```

```
collection = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2') \
```

```
    .filterDate('2023-01-01', '2023-12-31') \
```

```
    .limit(1)
```

```
print(f"📁 Found {collection.size().getInfo()} Landsat images")
```

```
except Exception as e:
```

```
    print(f"⚠️ GEE setup error: {e}")
```

```
    print("💡 Alternative: Use local data or pre-downloaded imagery")
```

5.2 GEE LULC Classification Function

```
python
```

```

def gee_lulc_classification(study_area, start_date, end_date):
    """
    Perform LULC classification using Google Earth Engine
    """
    try:
        # Load Sentinel-2 collection
        collection = ee.ImageCollection('COPERNICUS/S2_SR') \
            .filterBounds(study_area) \
            .filterDate(start_date, end_date) \
            .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))

        # Create median composite
        image = collection.median().clip(study_area)

        # Select bands for classification
        bands = ['B2', 'B3', 'B4', 'B8', 'B11', 'B12'] # Blue, Green, Red, NIR, SWIR1, SWIR2
        image = image.select(bands)

        print("📡 Satellite imagery prepared")
        print(f"📊 Using bands: {' '.join(bands)}")

        return image

    except Exception as e:
        print(f"❌ GEE processing error: {e}")
        return None

# Example study area (you would define your own)
# study_area = ee.Geometry.Rectangle([longitude1, latitude1, longitude2, latitude2])
print("📖 GEE classification function ready!")

```

Part 6: Evaluation and Visualization

6.1 Model Evaluation Functions

python

```

def evaluate_model(y_true, y_pred, class_names):
    """
    Comprehensive model evaluation
    """
    # Accuracy
    accuracy = accuracy_score(y_true, y_pred)

    # Classification report
    report = classification_report(y_true, y_pred,
                                   target_names=class_names,
                                   output_dict=True)

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)

    return accuracy, report, cm

def plot_confusion_matrix(cm, class_names):
    """
    Plot confusion matrix heatmap
    """
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names,
                yticklabels=class_names)
    plt.title('Confusion Matrix - LULC Classification')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)
    plt.tight_layout()
    plt.show()

def plot_training_history(train_losses, val_losses, train_accs, val_accs):
    """
    Plot training history for deep learning models
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # Loss plot
    ax1.plot(train_losses, label='Training Loss', color='blue')
    ax1.plot(val_losses, label='Validation Loss', color='red')
    ax1.set_title('Model Loss')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Loss')
    ax1.legend()

```

```
ax1.grid(True)
```

```
# Accuracy plot
```

```
ax2.plot(train_accs, label='Training Accuracy', color='blue')
```

```
ax2.plot(val_accs, label='Validation Accuracy', color='red')
```

```
ax2.set_title('Model Accuracy')
```

```
ax2.set_xlabel('Epoch')
```

```
ax2.set_ylabel('Accuracy')
```

```
ax2.legend()
```

```
ax2.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
print("📈 Evaluation and visualization functions ready!")
```

6.2 Results Visualization

```
python
```

```
def visualize_classification_results(original_image, predicted_map, class_names):
    """
    Visualize classification results
    """

    fig, axes = plt.subplots(1, 3, figsize=(18, 6))

    # Original image
    axes[0].imshow(original_image)
    axes[0].set_title('Original Satellite Image')
    axes[0].axis('off')

    # Predicted classification map
    im = axes[1].imshow(predicted_map, cmap='tab10', vmin=0, vmax=len(class_names)-1)
    axes[1].set_title('LULC Classification Map')
    axes[1].axis('off')

    # Color legend
    axes[2].axis('off')
    colors = plt.cm.tab10(np.linspace(0, 1, len(class_names)))
    for i, (color, class_name) in enumerate(zip(colors, class_names)):
        axes[2].scatter([], [], c=[color], s=100, label=class_name)
    axes[2].legend(loc='center', frameon=False, title='LULC Classes')

    plt.tight_layout()
    plt.show()

print("🤖 Visualization functions ready!")
```

Part 7: Example Workflow - Complete Pipeline

7.1 Demo with Synthetic Data

python

```
# Create synthetic data for demonstration
```

```
def create_demo_data(num_samples=1000, image_size=64, num_classes=5):
```

```
    """
```

```
    Create synthetic satellite-like data for demonstration
```

```
    """
```

```
    print("🔧 Creating synthetic demonstration data...")
```

```
    X = []
```

```
    y = []
```

```
    for class_id in range(num_classes):
```

```
        for _ in range(num_samples // num_classes):
```

```
            # Create synthetic "satellite" image
```

```
            # Each class has different spectral characteristics
```

```
            if class_id == 0: # Water
```

```
                image = np.random.normal(0.1, 0.05, (image_size, image_size, 3))
```

```
                image[:, :, 2] *= 1.5 # Higher blue
```

```
            elif class_id == 1: # Vegetation
```

```
                image = np.random.normal(0.3, 0.1, (image_size, image_size, 3))
```

```
                image[:, :, 1] *= 2.0 # Higher green
```

```
            elif class_id == 2: # Urban
```

```
                image = np.random.normal(0.5, 0.15, (image_size, image_size, 3))
```

```
            elif class_id == 3: # Bare soil
```

```
                image = np.random.normal(0.4, 0.1, (image_size, image_size, 3))
```

```
                image[:, :, 0] *= 1.3 # Higher red
```

```
            else: # Agriculture
```

```
                image = np.random.normal(0.25, 0.08, (image_size, image_size, 3))
```

```
                image[:, :, 1] *= 1.7 # Moderate green
```

```
            # Clip values to valid range
```

```
            image = np.clip(image, 0, 1)
```

```
            X.append(image)
```

```
            y.append(class_id)
```

```
    return np.array(X), np.array(y)
```

```
# Create demo dataset
```

```
demo_class_names = ['Water', 'Forest', 'Urban', 'Bare Soil', 'Agriculture']
```

```
X_demo, y_demo = create_demo_data(num_samples=500, num_classes=len(demo_class_names))
```

```
print(f"📊 Demo dataset created:")
```

```
print(f"  Images: {X_demo.shape}")
```

```
print(f"  Labels: {y_demo.shape}")
```

```
print(f"  Classes: {demo_class_names}")
```

7.2 Train and Evaluate Models

```
python

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X_demo, y_demo, test_size=0.3, random_state=42, stratify=y_demo
)

print("🔄 Training machine learning models...")

# Extract features for traditional ML
print(" Extracting features...")
X_train_features = np.array([extract_features(img) for img in X_train])
X_test_features = np.array([extract_features(img) for img in X_test])

# Train and evaluate traditional ML models
results = {}
for name, model in models.items():
    print(f" Training {name}...")
    model.fit(X_train_features, y_train)
    y_pred = model.predict(X_test_features)
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy
    print(f" {name} Accuracy: {accuracy:.3f}")

# Display results
print("\n✅ Model Comparison Results:")
for name, accuracy in results.items():
    print(f" {name}: {accuracy:.1%}")
```

7.3 Visualization of Results

```
python
```

```

# Plot model comparison
plt.figure(figsize=(10, 6))
names = list(results.keys())
accuracies = list(results.values())

bars = plt.bar(names, accuracies, color=['lightblue', 'lightcoral'])
plt.title('LULC Classification Model Comparison')
plt.ylabel('Accuracy')
plt.ylim(0, 1)

# Add value labels on bars
for bar, acc in zip(bars, accuracies):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
             f'{acc:.1%}', ha='center', va='bottom')

plt.tight_layout()
plt.show()

# Show confusion matrix for best model
best_model_name = max(results, key=results.get)
best_model = models[best_model_name]
y_pred_best = best_model.predict(X_test_features)

cm = confusion_matrix(y_test, y_pred_best)
plot_confusion_matrix(cm, demo_class_names)

print(f"🏆 Best performing model: {best_model_name}")

```

Part 8: Next Steps and Resources

8.1 Accessing Real Datasets

python

```

print("📚 Ready to work with real data? Try these resources:")
print()
print("🎯 Datasets:")
print("  1. EuroSAT: https://github.com/phelber/eurosat")
print("  2. UC Merced: http://weegee.vision.ucmerced.edu/datasets/landuse.html")
print("  3. Sentinel2GlobalLULC: https://zenodo.org/record/4172871")
print("  4. BigEarthNet: https://bigearth.net/")
print()
print("🔧 Advanced Tools:")
print("  1. Google Earth Engine: https://earthengine.google.com/")
print("  2. Sentinel Hub: https://www.sentinel-hub.com/")
print("  3. USGS Earth Explorer: https://earthexplorer.usgs.gov/")
print()
print("📖 Tutorials & Courses:")
print("  1. Climate Change AI LULC: https://github.com/climatechange-ai-tutorials/lulc-classification")
print("  2. GEE Python API: https://developers.google.com/earth-engine/guides")
print("  3. Spatial Thoughts GEE Course: https://courses.spatialthoughts.com/")

```

8.2 Improving Your Models

```

python

print("🚀 Tips for Better LULC Classification:")
print()
print("📊 Data Quality:")
print("  • Use cloud-free imagery")
print("  • Consider seasonal variations")
print("  • Ensure representative training samples")
print("  • Balance classes in training data")
print()
print("🧠 Model Improvements:")
print("  • Try ensemble methods")
print("  • Use transfer learning with pre-trained models")
print("  • Experiment with different architectures (ResNet, EfficientNet)")
print("  • Add data augmentation")
print()
print("🔍 Feature Engineering:")
print("  • Include spectral indices (NDVI, NDWI, etc.)")
print("  • Add texture features")
print("  • Use multi-temporal data")
print("  • Consider topographic features")

```

Part 9: Save Your Work

```
python
```

```

# Save trained models
import joblib
import torch

# Save traditional ML model
joblib.dump(best_model, 'best_lulc_model.pkl')
print("💾 Traditional ML model saved as 'best_lulc_model.pkl'")

# Save deep learning model (if trained)
# torch.save(model.state_dict(), 'lulc_cnn_model.pth')
# print("💾 Deep learning model saved as 'lulc_cnn_model.pth'")

# Save results to CSV
results_df = pd.DataFrame([
    {'Model': name, 'Accuracy': acc}
    for name, acc in results.items()
])
results_df.to_csv('lulc_results.csv', index=False)
print("📊 Results saved as 'lulc_results.csv'")

print("\n✅ Tutorial completed successfully!")
print("🎯 You now have the foundation for LULC classification using:")
print(" • Traditional machine learning (Random Forest, SVM)")
print(" • Deep learning (CNNs)")
print(" • Google Earth Engine integration")
print(" • Model evaluation and visualization")

```

Conclusion

This tutorial provides a comprehensive foundation for Land Use Land Cover classification using Google Colab. You can now:

- Set up a complete LULC classification pipeline
- Compare traditional ML vs. deep learning approaches
- Integrate with Google Earth Engine for large-scale analysis
- Evaluate and visualize your results effectively

Next Steps:

1. Try with real satellite imagery datasets
2. Experiment with more complex architectures
3. Scale up to larger geographic areas
4. Integrate temporal analysis for change detection

Happy classifying! 🌍📊